

A Library in a Droplet

How Digital Files Can Be Written into DNA

George M. Church¹ and Nick Goldman²

¹Harvard Medical School, Boston, USA

²European Bioinformatics Institute, Hinxton, UK

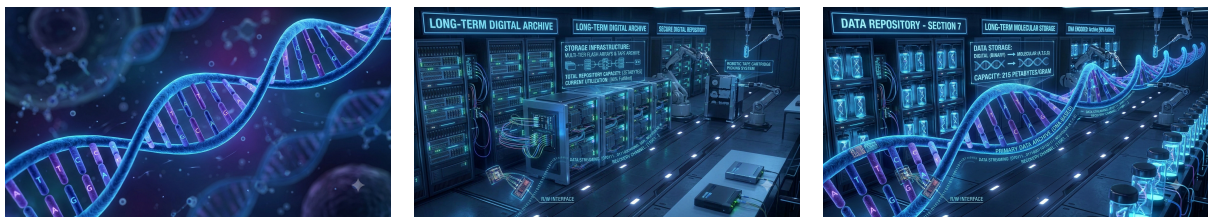
July 28, 2026

1 Can a File Become a Molecule?

What would a library look like if its pages were replaced by molecules? Instead of shelves and hard drives, imagine a tiny tube containing millions of invisible DNA strands. The idea sounds like science fiction, yet researchers have already stored books, images, audio, software, and other digital files in synthetic DNA [1].

DNA normally carries biological instructions, but its four-letter alphabet can also carry arbitrary computer data. The DNA need not be placed inside a living organism: it may be synthesized, dried, stored, and later read by a sequencing machine. The central idea is simple:

Convert bits into DNA letters, preserve the molecules, and reverse the conversion when the file is needed again.



(a) DNA as a biological molecule. (b) Digital data represented in bits. (c) DNA used as a storage medium.



The write path goes left to right; reading reverses the process.

(d) A simplified DNA data-storage cycle, from a digital file to synthetic DNA and back again.

Figure 1: From biological DNA to a complete DNA-based digital storage system.

The six stages in fig. 1d separate the electronic world from the molecular one. The computer performs the encoding and decoding; laboratory instruments perform the writing and reading.

1.1 A Four-Letter Storage Medium

Ordinary digital files are sequences of bits, each equal to 0 or 1. DNA uses four symbols:

$$\mathcal{A} = \{A, C, G, T\}$$

Table 1: One possible conversion from binary pairs to DNA bases.

Binary pair	DNA base
00	A
01	C
10	G
11	T

Because four symbols can represent four binary pairs,

$$4^n = (2^2)^n = 2^{2n} \quad (1)$$

so an ideal DNA symbol can carry two bits of information. Table 1 gives one possible toy mapping. Other mappings are equally valid. For example, the character A has binary code 01000001 in ASCII.¹ Grouping the bits in pairs and using Table 1 gives

$$A \rightarrow 01 \ 00 \ 00 \ 01 \rightarrow \text{CAAC} \quad (2)$$

This example explains the principle, but a real DNA-storage system needs more structure than a direct translation.

2 Building a Molecular File

Long files are divided into many short DNA strands called oligonucleotides, or oligos. Each oligo contains not only data but also information that helps the system locate and reconstruct it.



If the corresponding lengths are L_p, L_a, L_d , and L_c , then the total strand length is

$$L = 2L_p + L_a + L_d + L_c \quad (3)$$

The complete storage cycle can be summarized in five steps:

- I. **Encoding:** convert a file into carefully designed DNA sequences.
- II. **Synthesis:** manufacture those sequences as physical molecules.
- III. **Preservation:** keep the DNA dry, cold, or chemically protected [2].
- IV. **Sequencing:** read many copies of the stored strands.
- V. **Reconstruction:** order the fragments, correct errors, and rebuild the original file.

Primer regions can also support random access: instead of reading an entire DNA pool, a user can request one particular file. In one large experiment, researchers stored more than 200 MB across 35 files and recovered each file individually [3].

3 Addresses, Payload, and Useful Capacity

Suppose a file is divided among M DNA strands. To assign a distinct address to every strand, the required address length is

$$L_a = \lceil \log_4 M \rceil, \quad (4)$$

because an address of L_a DNA bases can represent 4^{L_a} different positions.

Using the strand structure in eq. 3, the number of bases available for actual file data is

$$L_d = L - 2L_p - L_a - L_c \quad (5)$$

¹ASCII is a standard numerical representation for common text characters.

Since one DNA base can ideally represent two bits, a file containing B bits requires approximately

$$M = \left\lceil \frac{B}{2L_d} \right\rceil \quad (6)$$

DNA strands. Thus, longer addresses and stronger error protection improve reliability, but leave less room for the data payload.

4 A Compact Encoding Procedure

The following procedure converts a bit string into addressed DNA strands. Here, $\text{Enc}(\cdot)$ applies the mapping in Table 1, and $\parallel$ denotes concatenation.

Algorithm 1 Create addressed DNA strands

Require: Bit string x , payload size q , and primers P_L, P_R

- 1: **procedure** CREATEADDRESSEDSTRANDS(x, q, P_L, P_R)
- 2: **for** each q -bit block x_i **do**
- 3: $a_i \leftarrow$ base-4 address of i , $d_i \leftarrow \text{Enc}(x)$
- 4: $c_i \leftarrow \text{Checksum}(a_i \parallel d_i)$
- 5: $s_i \leftarrow P_L \parallel a_i \parallel d_i \parallel c_i \parallel P_R$
- 6: **end for**
- 7: **return** $s_1, s_2, \dots, s_M$
- 8: **end procedure**

5 Why the Obvious Code Is Not Enough

The direct mapping in Table 1 is useful for learning, but biological hardware does not treat every DNA string equally well

5.1 Sequence Constraints

Three common design concerns are:

- ◊ Long repetitions: strings such as AAAAAAA are difficult to synthesize and read accurately.
- ◊ Unbalanced composition: strands with extremely high or low proportions of G and C may behave poorly in laboratory procedures.
- ◊ Lost ordering: millions of strands are mixed together, so each fragment needs an address.

For a DNA string s , its GC percentage is

$$\text{GC}(s) = \frac{N_G(s) + N_C(s)}{|s|} \times 100\% \quad (7)$$

where $N_G(s)$ and $N_C(s)$ count the two bases. Real encoders search for strings that carry the required bits while avoiding problematic patterns.

5.2 Errors at Different Stages

A stored strand can be altered in several ways:

Original AC GTAC
Substitution AC CTAC
Insertion AC GTTAC
Deletion ACTAC

Table 2: Where DNA-storage errors arise and how systems respond.

Phase	Stage	Challenge and response		
		Problem	Effect	Response
Writing	Encoding Synthesis	Repeats or extreme GC content	Poor synthesis or reading	Choose another sequence
		Base errors	INcorrect stand	Add checks and copoies
Preservation	Storage	Damage or strand loss	Missing fragments	Protect DNA and keep copies

6 Redundancy Without the Heavy Machinery

Error-correcting codes can become mathematically sophisticated, but their basic purpose is easy to see: store a little extra information so that imperfect reads do not destroy the message.

Suppose three sequencing reads report:

Read 1:ACGTAC

Read 2:ACGTAC

Read 3:ACCTAC

A position-by-position majority vote repairs the single disagreement. Real systems combine this idea with checksums, overlapping fragments, and stronger codes. The DNA Fountain system, for example, stored about 2.14 MB of digital data and recovered it perfectly using a robust coding architecture [4].

The extra DNA is not wasted space; it is insurance against missing and corrupted molecules.

References

- [1] George M. Church, Yuan Gao, and Sriram Kosuri. “Next-Generation Digital Information Storage in DNA”. In: *Science* 337.6102 (2012), p. 1628. DOI: [10.1126/science.1226355](https://doi.org/10.1126/science.1226355).
- [2] Yaniv Erlich and Dina Zielinski. “DNA Fountain Enables a Robust and Efficient Storage Architecture”. In: *Science* 355.6328 (2017), pp. 950–954. DOI: [10.1126/science.aaj2038](https://doi.org/10.1126/science.aaj2038).
- [3] Robert N. Grass et al. “Robust Chemical Preservation of Digital Information on DNA in Silica with Error-Correcting Codes”. In: *Angewandte Chemie International Edition* 54.8 (2015), pp. 2552–2555. DOI: [10.1002/anie.201411378](https://doi.org/10.1002/anie.201411378).
- [4] Lee Organick, Siena Dumas Ang, Yuan-Jyue Chen, et al. “Random Access in Large-Scale DNA Data Storage”. In: *Nature Biotechnology* 36 (2018), pp. 242–248. DOI: [10.1038/nbt.4079](https://doi.org/10.1038/nbt.4079).